HDU IS LABORATORY

Laboratory Work No. 3

Computer Attacks

Padding Oracle Attack on CBC Encryption

Recovering plaintext and forging authenticated state through a padding-validity side channel.

VARIANT	3
STUDENT	zhuhe
HDU NUMBER	23320116
DATE	20 October 2026

Contents

1 Purpose . . . 3
2 Executive Summary . . . 3
3 Attack Profile and Preconditions . . . 3
4 Technical Mechanism . . . 4
5 Attacker Sequence and Demonstration . . . 5
6 Directed-Graph Model . . . 7
7 Loss Analysis . . . 8
8 Countermeasures . . . 9
9 Conclusion . . . 9
10 References . . . 10
Appendix A - Review Questions . . . 11

1 Purpose

The purpose of this work is to analyze one computer attack in enough detail that its mechanics, prerequisites, observable transitions, and business consequences can be explained without ambiguity. The selected attack is the **padding oracle attack** on systems that encrypt structured data with CBC mode and PKCS#7 padding. The attack is represented as a directed graph whose initial state is an idle system, whose intermediate states are attacker actions and system responses, and whose final states are concrete losses.

A controlled localhost service was built to demonstrate the attack rather than merely describe it. The service encrypts a role-bearing session string, exposes a distinguishable padding error, and accepts the encrypted value as authoritative state. The exploit recovers the plaintext without the key and then changes the first plaintext block from `role=user` to `role=root`. The final response grants the administrator view. This produces real evidence for the confidentiality and integrity paths in the graph.

2 Executive Summary

The padding oracle is a chosen-ciphertext side channel. CBC decryption computes $P_i = D_K(C_i)$ xor C_{i-1}. If an application reveals whether the decrypted bytes end with valid PKCS#7 padding, an attacker can modify the preceding block, test candidate byte values, and learn one byte of $D_K(C_i)$ per successful test. Repeating this process recovers the plaintext. The same recovered intermediate bytes allow the attacker to calculate a forged preceding block for chosen plaintext.

Aspect	Result
Attack class	Cryptographic side-channel; chosen-ciphertext and adaptive oracle attack
Weakness	CBC encryption without a prior integrity check; distinguishable padding failures
Main precondition	Attacker can submit modified ciphertext and observe a validity difference
Primary losses	Plaintext disclosure, controlled modification, authentication or authority bypass
Prevention	AEAD such as AES-GCM, or Encrypt-then-MAC with constant-time verification
Proof produced	2 CBC blocks recovered in 4,713 oracle requests; forged role accepted as root

3 Attack Profile and Preconditions

The attack targets a protocol or application design, not a mathematical break of AES. AES can remain strong while the composition of CBC, padding validation, error handling, and state acceptance leaks the protected data. This distinction is central: the cipher is not broken, but the system's behavior supplies information that the cipher was supposed to hide.

Item	Condition
Affected component	Server-side decryption of client-controlled ciphertext, typically a cookie, hidden form field, URL parameter, or encrypted token.
Cryptographic condition	CBC mode with PKCS#7 padding and no authenticated-encryption construction.
Oracle condition	A visible difference between valid and invalid padding, such as status code, error text, response length, redirect, or timing.
Attacker position	Network access to the endpoint, a captured ciphertext, and the ability to send many modified versions.
Knowledge	Block size, ciphertext boundaries, and usually a rough expectation of the plaintext format.
Typical impact	Confidentiality loss, ciphertext forgery, privilege escalation, session manipulation, or account lockout during high-volume probing.

Historical evidence. CVE-2010-3332 [4] described detailed ASP.NET decryption errors that allowed attackers to decrypt and modify ViewState and possibly forge cookies. CVE-2014-3566 [5], known as POODLE, used CBC padding behavior in SSL 3.0 to recover plaintext through a man-in-the-middle position. These cases show why the oracle is not a theoretical curiosity.

4 Technical Mechanism

CBC decryption is deterministic for each ciphertext block, but it is chained: the previous block is xor-ed with the decrypted block to produce plaintext. The initialization vector (IV) performs that role for the first plaintext block. An attacker who changes the preceding block therefore changes the corresponding plaintext block in a predictable way, even though the attacker does not know the key.

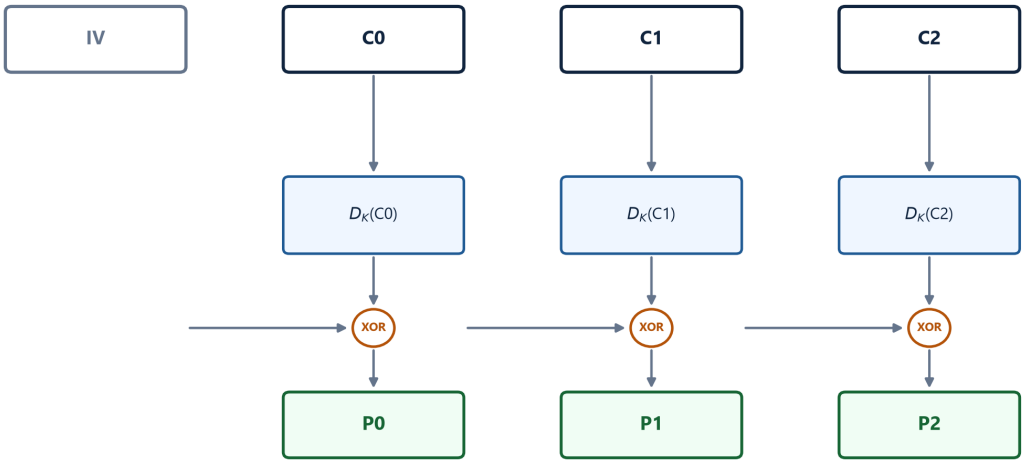


Figure 1. CBC decryption and the chaining value used by the oracle.

> **Core relation.** For block *i*: `P_i = D_K(C_i) xor C_(i-1)`. The attacker controls `C_(i-1)` and observes whether the resulting padding is valid. The unknown value `D_K(C_i)` is what the attack recovers one byte at a time.

4.1 PKCS#7 padding and the oracle

PKCS#7 pads a message to a multiple of the block size. With a 16-byte block, one missing byte is encoded as `01`, two missing bytes as `02 02`, and so on through sixteen missing bytes as sixteen `10` bytes. A decryptor checks the final byte and then checks that every padding byte has the same value. If the check fails, the normal result is a generic decryption failure.

The oracle exists when the attacker can tell whether that check passed. Passing validity is a single bit of information, but it is enough for adaptive queries. A response can be an explicit message such as `invalid padding`, a different HTTP status, a redirect, or a timing difference. The exact channel is less important than the fact that the attacker can separate valid from invalid padding reliably.

4.2 Recovering one byte

To recover the last byte of a block, the attacker changes the last byte of the preceding block and sends up to 256 candidates. The goal is a plaintext ending in padding value `01`. If candidate g produces valid padding, then $D_K(C_i)[15]$ xor $g = 1$, so the intermediate byte is $I[15] = g$ xor 1. After that byte is known, the attacker targets padding `02 02`, then `03 03 03`, and repeats until all sixteen bytes are known. The formula for every position is $I[j] = g$ xor pad.

A practical implementation must handle a false positive at the first byte: a candidate could create two valid padding bytes instead of one. The demonstration flips a second preceding byte during the first-byte search so that only a true one-byte pad is accepted. This is a small but important detail that distinguishes a working exploit from a toy loop.

The online cost is bounded by 256 guesses for each byte. For a 16-byte block, the worst case is 4,096 requests per block; with uniformly distributed bytes, the expected number is roughly half that per byte. The observed two-block run used 4,713 requests, which is consistent with the expected order of magnitude and includes the extra request for the oracle check.

4.3 Forging chosen plaintext

Once the attacker knows a block's intermediate value, no key is needed to make a controlled change. To obtain a target plaintext block *P**, the attacker calculates the preceding block as `C*_(i−1) = I_i xor P*_i`. For the first plaintext block, the predecessor is the IV, so changing the IV does not corrupt any earlier plaintext. For a later block, changing the preceding ciphertext also corrupts the previous plaintext block; a real exploit may need to account for that constraint, use a tolerant parser, or forge the preceding block as part of a chain. The demonstration deliberately targets the first block to isolate the integrity failure.

5 Attacker Sequence and Demonstration

The laboratory target is a local Flask service bound to 127.0.0.1. It uses a 256-bit AES key, a random IV, CBC mode, and PKCS#7 padding. The protected message is `role=user;uid=1001;ok=1`. The service intentionally returns `invalid padding` separately from a valid-padding payload parse error, and it trusts the decrypted role. No real user data or organization is involved.

Stage	Action
1. Acquire	Request a session and record the base64 value containing IV and ciphertext.
2. Confirm	Flip one ciphertext byte. A 400 invalid-padding response proves the oracle is observable.
3. Recover	For each block, submit modified predecessor bytes and search each position for valid padding.
4. Assemble	Xor the recovered intermediate block with the real predecessor to obtain plaintext.
5. Forge	Calculate a new IV from the recovered intermediate value and the chosen first plaintext block.
6. Verify	Submit the forged cookie and observe an authorized administrator response.

```
GET /session
POST /account  {"cookie": "<iv||ciphertext in base64>"}

# controls used by the exploit
probe = previous_block_with_one_byte_flipped + target_ciphertext_block
valid_padding = (response.status == 400 and error != "invalid padding") or response.status != 400
intermediate[position] = guess ^ padding_value
plaintext_block = intermediate xor original_previous_block
forged_iv = intermediate xor chosen_plaintext_block
```

Observed result. The service returned the original user dashboard, then returned `role=root` and `access=admin console` for the forged cookie. The forged value was not a replay: it contained a new IV and a different plaintext block. The exploit's 4,713 requests demonstrate the online nature of the attack and its dependence on an observable oracle.

5.1 Recorded evidence

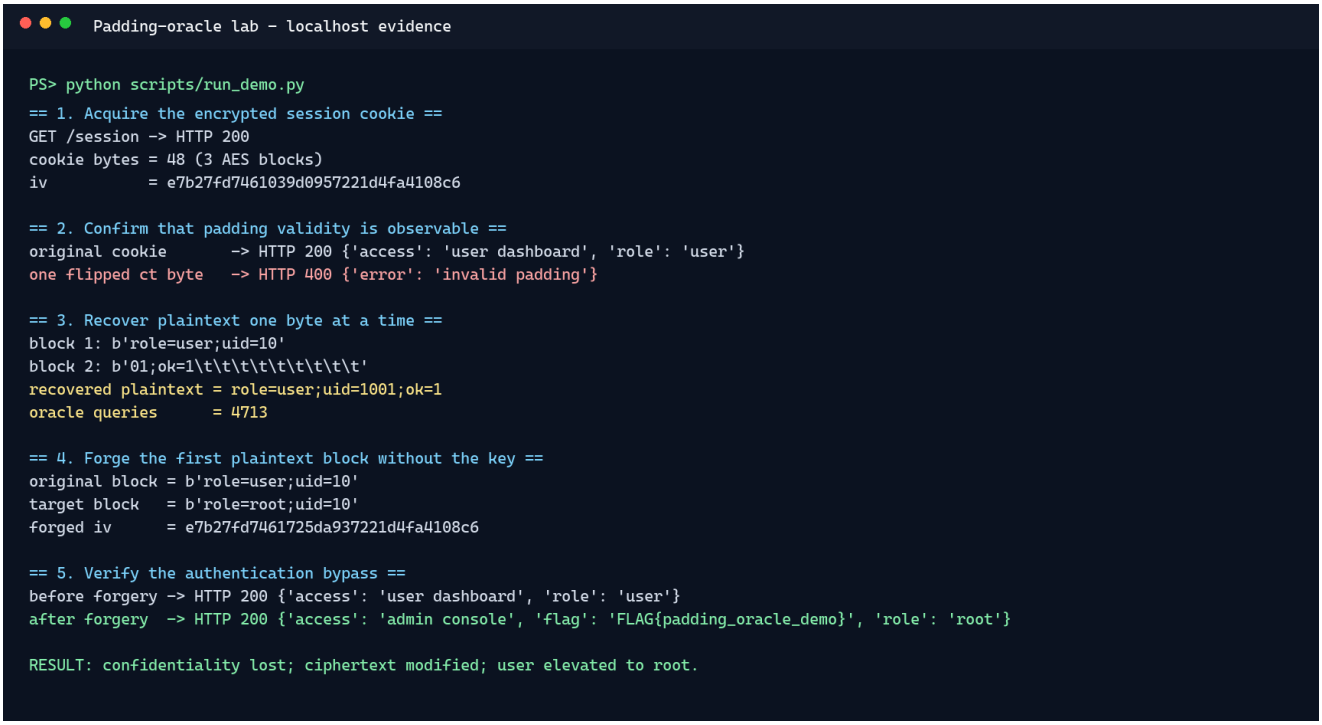


Figure 2. Local execution: plaintext recovery and privilege change from user to root.

Evidence line	Graph state	Interpretation
original cookie -> HTTP 200	S0-S1	The baseline session is valid and returns the user dashboard.
flipped byte -> HTTP 400 invalid padding	S2-S3	The oracle is observable through a distinct padding error.
block 1 and block 2 output	S4-S6	The intermediate blocks are recovered and converted to plaintext.
recovered plaintext line	S7-K1	Confidentiality is lost without possession of the AES key.
after forgery -> role=root	S8-S9-K2	The modified IV is accepted and the authorization decision changes.

The run used 4,713 HTTP calls against a loopback service. That number is not a performance benchmark; it shows that the attack is an online, adaptive process. A real target may require parallelism, connection reuse, or timing analysis, and defensive rate limits can change the practical cost. The demonstration therefore supports the logical graph rather than claiming universal timing or exploitability.

The service and exploit are contained in the accompanying repository. The target binds only to 127.0.0.1, uses synthetic data, and must not be exposed to a network. The evidence was produced by the same code shown in the report, so the recovered blocks and the forged response are reproducible rather than hand-written narrative.

6 Directed-Graph Model

The model is a finite directed graph $G = (V, E)$. The vertex set begins with the idle state S_0, continues through attacker and system states S_i, and terminates in one of the loss states S_k. Each edge is labelled with the action or observable response that causes the transition. The self-loop on S3 represents repeated guesses and repeated block processing.

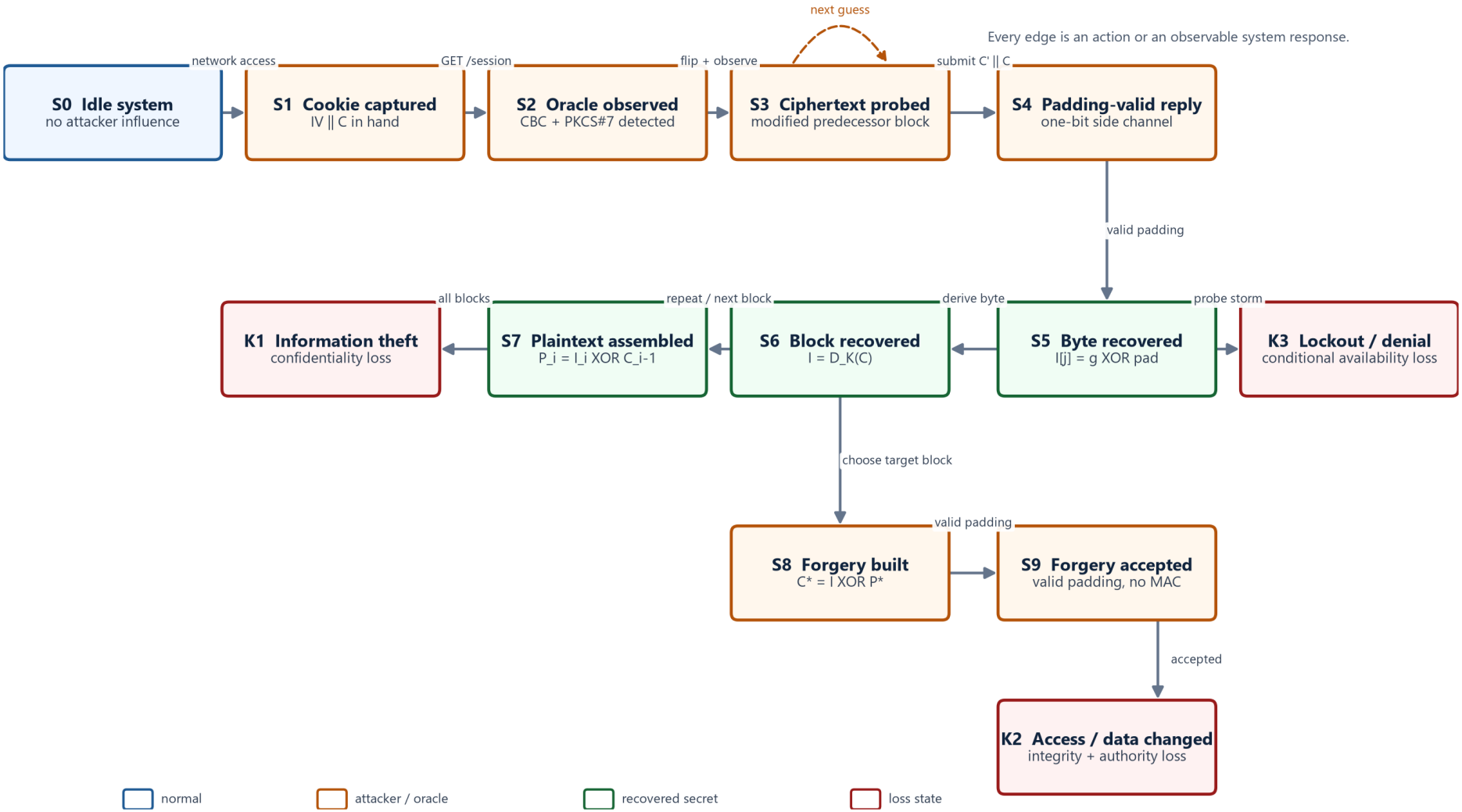


Figure 3. Directed graph of the padding oracle attack. Blue is the baseline, orange is attacker or oracle activity, green is recovered secret material, and red is a final loss state.

The three terminal paths are intentional: K1 is confidentiality loss, K2 is integrity and authority loss, and K3 is a conditional availability loss caused by lockouts or resource pressure. K3 is secondary; not every padding oracle attack produces denial of service.

6.1 State definitions

State	Meaning in the system	Entry condition
S0	System idle; no attacker influence; legitimate encrypted state exists.	Baseline
S1	Attacker captures IV || C, for example from a cookie or hidden field.	Network access and observation
S2	Attacker identifies CBC with PKCS#7 and detects a padding-validity difference.	Flipped-byte test; error or timing
S3	Attacker submits a modified predecessor block and a target ciphertext block.	Controlled ciphertext modification
S4	Oracle returns a padding-valid response for a candidate byte.	One-bit side-channel response
S5	Attacker derives one intermediate byte I[j] = g xor pad and moves to the next position.	Adaptive query loop
S6	All 16 bytes of a block are recovered; I = D_K(C).	Sixteen successful positions
S7	Plaintext is assembled using P_i = I_i xor predecessor.	Confidentiality impact
K1	Attacker has the plaintext, session content, or another protected value.	Information theft
S8	Attacker calculates C* = I xor P* for a chosen plaintext block.	Forgery construction
S9	Server validates the forged ciphertext and accepts the altered state.	No MAC or tag was checked
K2	Attacker changes data, impersonates a role, or obtains unauthorized access.	Integrity and authority loss
K3	Repeated probing, lockout, or resource pressure denies access to a legitimate user.	Conditional availability loss

6.2 Transition and path consistency

Transition	Justification
S0 -> S1	Attacker obtains network access and captures an encrypted value.
S1 -> S2	The response is inspected; a modified ciphertext produces a distinguishable padding failure.
S2 -> S3	The attacker selects a target block and begins controlled modifications.
S3 -> S3	Next candidate, next byte, or next block; the query count grows but the state remains in the loop.
S3 -> S4	A candidate produces a padding-valid response.
S4 -> S5	The intermediate byte is calculated from the successful guess and target padding.
S5 -> S6	The block is complete after all positions are recovered.
S6 -> S7 -> K1	The recovered intermediate block is xor-ed with the real predecessor; plaintext is assembled.
S6 -> S8 -> S9 -> K2	The attacker chooses a target, builds a predecessor, and submits a forged value.
S3 -> K3	Only when probing causes a lockout, quota, or resource collapse; this is not guaranteed.

Path verification. The confidentiality path is S0-S1-S2-S3-S4-S5-S6-S7-K1. The integrity path is S0-S1-S2-S3-S4-S5-S6-S8-S9-K2. The conditional availability path is S0-S1-S2-S3-K3. The S3 self-loop is not a new endpoint; it repeats the guessing action until S4 is reached. Every non-loop edge has a concrete action or system response, and no terminal loss is reachable without a prior oracle observation.

7 Loss Analysis

Final state	Asset affected	Loss mechanism
K1 - Information theft	Confidential plaintext, session claims, identifiers, or business data	The attacker recovers block contents and reconstructs the full message.
K2 - Modification or unauthorized access	Integrity of state and authority of the session	The attacker forges a role-bearing block and the service accepts it as root.
K3 - Denial of service or access	Availability and responsiveness	High-volume oracle attempts can trigger lockouts, throttling, or resource exhaustion.

The demonstration directly realizes K1 and K2. It does not claim that a padding oracle always causes a denial of service: K3 depends on the surrounding controls and traffic volume. This is why the graph labels K3 as conditional. The distinction keeps the model defensible: a path is shown only when the system state can actually transition there.

The business impact is broader than one disclosed string. Encrypted cookies often contain tenant identifiers, user IDs, feature flags, or serialized state. A forged block can change authorization decisions, while leaked identifiers can support follow-on reconnaissance and targeted attacks. Because the exploit is online and can be automated, a single weak endpoint can produce many recovered sessions or repeated privilege changes.

8 Countermeasures

The correct remediation is not 'hide the error message' in isolation. Error uniformity makes the oracle harder to observe, but the application still lacks a cryptographic integrity check. A safe design verifies authenticity before it decrypts, parses, or trusts any plaintext. Prevention should be layered so that one missing control does not reopen the entire path.

Layer	Action	Why it works
Primary	Use an AEAD construction such as AES-GCM or ChaCha20-Poly1305 and verify the tag before parsing.	Removes the unauthenticated CBC state and turns modification into a hard failure.
Compatibility	If CBC must remain, use Encrypt-then-MAC with HMAC-SHA-256 over IV || ciphertext and compare in constant time.	Rejects forged predecessors before the padding check can become an oracle.
Failure handling	Return one generic error for malformed, invalid-tag, and invalid-padding cases; avoid secret-dependent branches and timing leaks.	Reduces the observable distinction even during migration.
Operations	Rate-limit decryption failures, alert on repeated tampering, rotate keys, invalidate old cookies, and add regression tests that flip every ciphertext byte.	Limits automated probing and removes previously captured credentials after a fix.

8.1 Verification pattern

```
# Preferred: authenticated encryption. The tag check happens before parse.
plaintext = aes_gcm.decrypt(nonce, ciphertext_with_tag, associated_data)
session = parse_session(plaintext)  # only after successful authentication

# If CBC is unavoidable: Encrypt-then-MAC, then constant-time compare.
expected = hmac_sha256(mac_key, iv + ciphertext)
if not hmac.compare_digest(expected, received_tag):
    reject_with_generic_error()
plaintext = cbc_decrypt(key, iv, ciphertext)
session = parse_session(remove_pkcs7(plaintext))
```

The local lab should not be deployed. Its key, endpoint, and deliberately distinguishable padding response are educational artifacts. A production migration would also need to decide how to handle already-issued cookies: the safest choice is coordinated invalidation and reissuance, because an attacker may have captured encrypted values before the patch.

9 Conclusion

The padding oracle attack shows how a small observable difference becomes a complete confidentiality and integrity compromise. CBC and PKCS#7 are not inherently malicious, but they do not authenticate the ciphertext. Once the application treats padding validity as a user-visible signal and trusts decrypted state, an attacker can turn that signal into a byte-by-byte decryption oracle and then into a forgery primitive.

The directed graph makes the attack auditable. It fixes the baseline S0, names each intermediate action and response, exposes the self-loop that drives the query cost, and makes the K1, K2, and conditional K3 outcomes explicit. The most effective defense is equally structural: authenticate before decrypting, reject tampering before parsing, and remove the observable oracle rather than merely changing its error text.

> **Takeaway.** Encryption without authentication protects against reading ciphertext, not against modifying it. The padding oracle is a system-design failure, not evidence that AES has been broken.

10 References

[1] S. Vaudenay, "Security Flaws Induced by CBC Padding - Applications to SSL, IPSEC, WTLS...," EUROCRYPT 2002. https://doi.org/10.1007/3-540-46035-7_35

[2] NIST, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," SP 800-38A. https://doi.org/10.6028/NIST.SP.800-38A

[3] NIST, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," SP 800-38D. https://doi.org/10.6028/NIST.SP.800-38D

[4] NVD, "CVE-2010-3332 - ASP.NET Padding Oracle Vulnerability." https://nvd.nist.gov/vuln/detail/CVE-2010-3332

[5] NVD, "CVE-2014-3566 - POODLE." https://nvd.nist.gov/vuln/detail/CVE-2014-3566

[6] OWASP, "Cryptographic Storage Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

[7] OWASP, "A02:2021 - Cryptographic Failures," OWASP Top 10. https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

[8] J. Schaad et al., "An Interface and Algorithms for Authenticated Encryption," RFC 5116. https://www.rfc-editor.org/rfc/rfc5116

[9] MITRE, "CWE-353: Missing Support for Integrity Check." https://cwe.mitre.org/data/definitions/353.html

[10] MITRE ATT&CK, "T1557: Adversary-in-the-Middle." https://attack.mitre.org/techniques/T1557/

Appendix A - Review Questions

1. Vulnerability versus exploit

A vulnerability is the weakness itself: unauthenticated CBC encryption plus a distinguishable padding check. An exploit is the concrete method that turns the weakness into an outcome. The digraph primarily models the exploit path through system states, while the missing integrity check and observable error are the vulnerability conditions that make the path possible.

2. Why S0 and Sk must be explicit

S0 defines the baseline against which all attacker influence is measured. The final states define what counts as a loss. Without both, a diagram can become an unordered collection of attack steps and cannot demonstrate that every path ends in a concrete consequence or that each transition is justified.

3. Different digraphs for the same attack

A decrypt-only strategy reaches K1 without S8 or S9. A privilege-escalation strategy includes forgery and acceptance. An attacker who targets a later block may add states for repairing or cascading corruption in preceding blocks. A timing-based oracle may replace explicit error states with repeated measurements. The graph changes with strategy, target, and available observation.

4. Loss categories

This attack directly causes information theft and modification or unauthorized access. It can cause denial of service or access when probing triggers lockout or resource exhaustion, but that path is conditional. The demonstration provides evidence for theft and privilege change, not a guaranteed denial of service.

5. Value of the digraph for countermeasures

The graph identifies choke points. If the oracle edge S3-S4 is removed, neither K1 nor K2 is reachable through the demonstrated route. If integrity is verified before S9, the forged block cannot be accepted. The graph therefore converts a narrative into testable security properties and makes clear that a control must block all relevant paths, not only one outcome.